Министерство образования и исследований
Государственный университет Молдовы
Отдел математики и информатики

ОТЧЕТ

по лабораторной работе №2

«Реализация базовых методов работы с массивами»

Выполнил: студент группы IA2503ru
Антонович Б.
Проверил: Калин Н.

2026

1 Теоретическая часть

1.1 Формулировка задачи

Задачей данной лабораторной работы является реализация базовых функций для работы с массивами: forEach, map, filter, find, some, every и reduce. При этом использование встроенных методов массивов запрещено — разрешено применять только цикл for, свойство length, обращение к элементам по индексу и метод push. Код должен быть задокументирован с использованием стандарта JSDoc.

1.2 Описание цели и основные этапы работы

Цель работы: Изучить принципы реализации базовых методов работы с массивами с использованием функций обратного вызова (колбэков) и закрепить навыки написания функций с корректной обработкой граничных случаев.

Основные этапы работы:

- Реализация вспомогательных функций printArray и printArray1.
- Реализация универсальной функции forEach.
- Реализация функций map и filter для преобразования и фильтрации.
- Реализация функций find, some, every для поиска и проверки условий.
- Реализация функции reduce для накопительной обработки элементов.

2 Практическая часть

Задание 1. Вспомогательные функции printArray и printArray1

Для демонстрации необходимости универсального подхода реализованы две функции вывода массива в разных форматах с использованием цикла for.

```
C: > Users > bdm > Labs > vscod > JS > lab2 > JS lab2.js > forEach
function printArray(array) {
  for (let i = 0; i < array.length; i++) {
    console.log("Element " + i + ": value " + array[i]);
  }
}

function printArray1(array) {
  for (let i = 0; i < array.length; i++) {
    console.log(i + ": " + array[i]);
  }
}
```

Наличие двух практически одинаковых функций наглядно показывает проблему дублирования кода. Обе содержат одинаковый цикл обхода — отличается лишь формат вывода. Это и является мотивацией для реализации универсальной функции forEach.

```
Element 0: value 1
Element 1: value 2
Element 2: value 3
Element 3: value 4
Element 4: value 5
0: 1
1: 2
2: 3
3: 4
4: 5
```

Задание 2. Функция forEach(array, callback)

Функция обходит каждый элемент массива и вызывает переданный колбэк с тремя аргументами: текущим элементом, его индексом и всем массивом. Функция ничего не возвращает.

```
function forEach(array, callback) {
  for (let i = 0; i < array.length; i++) {
    callback(array[i], i, array);
  }
}

forEach([1, 2, 3], function(element, index) {
  console.log("Element: " + element + ", Index: " + index);
});
```

Результат выполнения:

```
Element: 1, Index: 0
Element: 2, Index: 1
Element: 3, Index: 2
```

Задание 3. Функция map(array, callback)

Функция создаёт новый массив той же длины, где каждый элемент — результат вызова колбэка. Исходный массив не изменяется.

```
function map(array, callback) {
  let result = [];
  for (let i = 0; i < array.length; i++) {
    result.push(callback(array[i], i, array));
  }
  return result;
}

let squared = map([1, 2, 3], function(element) {
  return element * element;
});
console.log(squared);
```

```
> (3) [1, 4, 9]
```

Задание 4. Функция filter(array, callback)

Функция формирует новый массив, включая только те элементы, для которых колбэк вернул true.

```
function filter(array, callback) {
  let result = [];
  for (let i = 0; i < array.length; i++) {
    if (callback(array[i], i, array)) {
      result.push(array[i]);
    }
  }
  return result;
}

let evenNumbers = filter([1, 2, 3, 4, 5], function(element) {
  return element % 2 === 0;
});
console.log(evenNumbers);
```

```
> (2) [2, 4]
```

Задание 5. Функция find(array, callback)

Функция возвращает первый элемент, для которого колбэк вернул true. Если совпадение не найдено — возвращает undefined. Обход прекращается при первом совпадении.

```
function find(array, callback) {
  for (let i = 0; i < array.length; i++) {
    if (callback(array[i], i, array)) {
      return array[i];
    }
  }
  return undefined;
}

let firstEven = find([1, 2, 3, 4, 5], function(element) {
  return element % 2 === 0;
});
console.log(firstEven);
```

```
2
```

Задание 6. Функция some(array, callback)

Функция проверяет, существует ли хотя бы один элемент, удовлетворяющий условию. Возвращает true при первом совпадении, иначе — false.

```
function some(array, callback) {
  for (let i = 0; i < array.length; i++) {
    if (callback(array[i], i, array)) {
      return true;
    }
  }
  return false;
}

let hasEven = some([1, 2, 3, 4, 5], function(element) {
  return element % 2 === 0;
});
console.log(hasEven);
```

```
true
```

Задание 7. Функция every(array, callback)

Функция проверяет, удовлетворяют ли все элементы заданному условию. Возвращает false при первом несоответствии, не продолжая обход.

```
function every(array, callback) {
  for (let i = 0; i < array.length; i++) {
    if (!callback(array[i], i, array)) {
      return false;
    }
  }
  return true;
}

let allEven = every([2, 4, 6], function(element) {
  return element % 2 === 0;
});
console.log(allEven);
```

```
true
```

Задание 8. Функция reduce(array, callback, initialValue)

Функция последовательно обрабатывает элементы, накапливая результат в аккумуляторе. Если initialValue не передано — используется первый элемент массива, а обход начинается со второго. Если массив пуст и initialValue не указано — возвращается undefined.

```
function reduce(array, callback, initialValue) {
  let acc;
  let start;

  if (initialValue === undefined) {
    acc = array[0];
    start = 1;
  } else {
    acc = initialValue;
    start = 0;
  }

  for (let i = start; i < array.length; i++) {
    acc = callback(acc, array[i], i, array);
  }
  return acc;
}

let sum = reduce([1, 2, 3, 4, 5], function(acc, element) {
  return acc + element;
}, 0);
console.log(sum);
```

```
15
```

Вывод всех функции:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  C:\Program Files\nodejs\node.exe .\lab2.js
  Element 0: value 1
  Element 1: value 2
  Element 2: value 3
  Element 3: value 4
  Element 4: value 5
  0: 1
  1: 2
  2: 3
  3: 4
  4: 5
  Element: 1, Index: 0
  Element: 2, Index: 1
  Element: 3, Index: 2
> (3) [1, 4, 9]
> (2) [2, 4]
  2
  true
  true
  15
```

3 Вывод и ссылка на репозиторий Git

В ходе выполнения лабораторной работы были успешно реализованы все базовые методы работы с массивами: forEach, map, filter, find, some, every и reduce — без использования встроенных методов JavaScript. Было наглядно показано, как колбэки позволяют устранить дублирование кода и создать универсальные функции. Полученные навыки являются основой для понимания функционального программирования в JavaScript.

Ссылка на репозиторий: https://github.com/badm1nton1/jslabs/tree/main/lab2

4 Ответы на контрольные вопросы

1. В чём преимущества использования колбэков при работе с массивами?

Колбэки удобны тем, что позволяют одной и той же функции работать по-разному в зависимости от задачи. Мы просто передаём внутрь функцию с нужным поведением, и не нужно каждый раз писать новый цикл. Это делает код короче, гибче и проще для изменения: если нужно поменять логику, достаточно изменить колбэк, а не переписывать всё заново.

2. Какие проблемы могут возникать при использовании колбэков и как их избежать?

Проблемы могут возникать, если колбэков слишком много или они слишком сложные — тогда код становится трудным для понимания. Также легко забыть вернуть значение или передать не ту функцию. Чтобы этого избежать, нужно писать колбэки простыми и короткими, давать понятные имена, не делать слишком глубокую вложенность и внимательно следить за тем, что функция возвращает.

3. Как реализовать функции map, filter, find, some, every и reduce без встроенных методов?

Функции map, filter, find, some, every и reduce можно реализовать без встроенных методов с помощью обычного цикла. Смысл в том, что мы проходим по массиву по очереди, берём каждый элемент и решаем, что с ним делать: в map — создаём новый изменённый массив, в filter — оставляем только подходящие элементы, в find — возвращаем первый подходящий, в some — проверяем, есть ли хотя бы один подходящий, в every — проверяем, подходят ли все, а в reduce — постепенно объединяем все элементы в одно значение, например сумму.

5 Список использованных источников

— Современный учебник JavaScript — https://learn.javascript.ru/

— MDN Web Docs — https://developer.mozilla.org/

— JSDoc Reference — https://jsdoc.app/